

ArqWeb — Gestor de Vulnerabilidades

Trabajo Práctico Integrador — Arquitectura Web

Alumno: Juan Carlos Perez Zabala **Repositorio:** <https://github.com/juancapz/ArqWeb>

1. Descripción del backend

El backend es una **API RESTful** desarrollada en **Node.js** con el framework **Express**, que implementa un sistema de gestión de vulnerabilidades de seguridad informática. El sistema permite registrar, consultar, modificar y dar seguimiento al ciclo de vida de vulnerabilidades detectadas en los activos (servidores, aplicaciones, bases de datos, etc.) de una organización, así como asignar responsables para su remediación.

Dominio del problema

El sistema modela un caso de uso real del área de ciberseguridad: los equipos de seguridad necesitan mantener un inventario centralizado de vulnerabilidades encontradas durante auditorías, pentests o reportes de terceros, clasificarlas por severidad, asociarlas a los activos afectados, asignarlas a un responsable y realizar seguimiento hasta su remediación. A partir de esos datos se generan reportes que permiten a los equipos priorizar esfuerzos y medir el estado de seguridad.

Entidades principales

- **Vulnerability** — Vulnerabilidad detectada. Incluye título, descripción, CVE asociado (opcional), severidad, score CVSS, estado del ciclo de vida, activo afectado, responsable asignado y fechas relevantes.
- **Asset** — Activo de la organización sobre el que puede existir una vulnerabilidad (servidor, aplicación, base de datos, dispositivo de red, endpoint). Incluye tipo, entorno (producción/staging/desarrollo), dueño y nivel de criticidad.
- **User** — Usuario responsable de la remediación de vulnerabilidades. Incluye nombre, email y rol.

Decisiones técnicas

- **Lenguaje:** JavaScript (Node.js).
- **Framework:** Express.

Estructura de la API

Todos los endpoints están prefijados con `/api`. Los recursos expuestos son:

- `/api/vulnerabilities` — Gestión de vulnerabilidades (recurso central del sistema).
 - `/api/assets` — Gestión de activos.
 - `/api/users` — Gestión de usuarios responsables.
 - `/api/reports` — Reportes derivados de los datos del sistema (solo lectura).
-

2. Documentación de la API

Códigos de estado utilizados

Código	Significado	Cuándo se devuelve
200 OK	Operación exitosa con contenido en la respuesta	GET exitoso, PUT exitoso
201 Created	Recurso creado correctamente	POST exitoso
204 No Content	Operación exitosa sin contenido en la respuesta	DELETE exitoso
400 Bad Request	El cuerpo o los parámetros son inválidos	Datos faltantes, tipos incorrectos, valores fuera de enum, ID mal formado
404 Not Found	El recurso solicitado no existe	ID inexistente en la ruta

2.1 Recurso: Vulnerabilities

GET /api/vulnerabilities

Lista todas las vulnerabilidades registradas. Admite filtros opcionales por query string: `status` , `severity` , `assetId` , `assignedTo` .

Ejemplo: `GET /api/vulnerabilities?status=Abierta&severity=Crítica`

Código	Descripción
200 OK	Devuelve un array de vulnerabilidades (vacío si no hay coincidencias)
400 Bad Request	Algún valor de query es inválido

GET /api/vulnerabilities/:id

Obtiene el detalle de una vulnerabilidad por su ID.

Código	Descripción
200 OK	Devuelve el objeto vulnerabilidad
400 Bad Request	El <code>:id</code> no es un entero positivo válido
404 Not Found	No existe una vulnerabilidad con ese ID

POST /api/vulnerabilities

Crea una nueva vulnerabilidad. El servidor asigna el ID. Campos requeridos: `title` , `severity` , `assetId` . Campos opcionales: `description` , `cve` , `cvssScore` , `assignedTo` , `status` (por defecto `Abierta`).

Código	Descripción
201 Created	Vulnerabilidad creada. Devuelve el objeto creado con su ID
400 Bad Request	Faltan campos requeridos, tipos incorrectos o valores fuera de los enums permitidos
404 Not Found	El <code>assetId</code> o <code>assignedTo</code> referenciado no existe

PUT /api/vulnerabilities/:id

Reemplaza completamente una vulnerabilidad existente. Requiere enviar todos los campos obligatorios del recurso.

Código	Descripción
200 OK	Vulnerabilidad actualizada. Devuelve el objeto actualizado
400 Bad Request	El body es inválido o incompleto
404 Not Found	No existe una vulnerabilidad con ese ID, o el <code>assetId</code> / <code>assignedTo</code> referenciado no existe

DELETE /api/vulnerabilities/:id

Elimina una vulnerabilidad.

Código	Descripción
204 No Content	Eliminación exitosa, sin cuerpo en la respuesta
400 Bad Request	El :id no es un entero positivo válido
404 Not Found	No existe una vulnerabilidad con ese ID

2.2 Recurso: Assets

GET /api/assets

Lista todos los activos. Admite filtros opcionales por query string: `type`, `environment`, `criticality`.

Ejemplo: `GET /api/assets?criticality=Alta&environment=Producción`

Código	Descripción
200 OK	Devuelve un array de activos
400 Bad Request	Algún valor de query es inválido

GET /api/assets/:id

Obtiene el detalle de un activo.

Código	Descripción
200 OK	Devuelve el objeto activo
400 Bad Request	El :id no es un entero positivo válido
404 Not Found	No existe un activo con ese ID

POST /api/assets

Crea un nuevo activo. Campos requeridos: `name`, `type`, `environment`, `criticality`. Campo opcional: `owner`.

Código	Descripción
201 Created	Activo creado. Devuelve el objeto creado con su ID
400 Bad Request	Faltan campos requeridos o hay valores inválidos

PUT /api/assets/:id

Reemplaza completamente un activo existente.

Código	Descripción
200 OK	Activo actualizado. Devuelve el objeto actualizado
400 Bad Request	El body es inválido o incompleto
404 Not Found	No existe un activo con ese ID

DELETE /api/assets/:id

Elimina un activo.

Código	Descripción
204 No Content	Eliminación exitosa
400 Bad Request	El :id no es un entero positivo válido
404 Not Found	No existe un activo con ese ID

2.3 Recurso: Users

GET /api/users

Lista todos los usuarios.

Código	Descripción
200 OK	Devuelve un array de usuarios

GET /api/users/:id

Obtiene el detalle de un usuario.

Código	Descripción
200 OK	Devuelve el objeto usuario
400 Bad Request	El :id no es un entero positivo válido
404 Not Found	No existe un usuario con ese ID

POST /api/users

Crea un nuevo usuario. Campos requeridos: `name`, `email`, `role`.

Código	Descripción
201 Created	Usuario creado
400 Bad Request	Faltan campos, tipos inválidos o email con formato incorrecto

PUT /api/users/:id

Reemplaza completamente un usuario.

Código	Descripción
200 OK	Usuario actualizado
400 Bad Request	El body es inválido o incompleto
404 Not Found	No existe un usuario con ese ID

DELETE /api/users/:id

Elimina un usuario.

Código	Descripción
204 No Content	Eliminación exitosa
400 Bad Request	El :id no es un entero positivo válido
404 Not Found	No existe un usuario con ese ID

2.4 Recurso: Reports

Endpoint de solo lectura que agrega información derivada de los datos del sistema.

GET /api/reports/severity-distribution

Devuelve el conteo de vulnerabilidades agrupadas por severidad.

Ejemplo de respuesta:

```
{
  "Crítica": 3,
  "Alta": 7,
  "Media": 12,
  "Baja": 5
}
```

Código	Descripción
200 OK	Devuelve el objeto con la distribución por severidad

3. Enums y valores permitidos

- **Vulnerability.severity:** Crítica, Alta, Media, Baja
- **Vulnerability.status:** Abierta, En progreso, Resuelta
- **Asset.type:** Servidor, Aplicación, Base de datos, Dispositivo de red, Endpoint
- **Asset.environment:** Producción, Staging, Desarrollo
- **Asset.criticality:** Alta, Media, Baja
- **User.role:** Analista de seguridad, Desarrollador, Administrador de sistemas